

Ziekenhuis Geel

Realisatiedocument

Steff Dierckx
Student Bachelor in de Toegepaste Informatica – Applicatieontwikkeling

Inhoudsopgave

1. INLEIDING	4
2. ALGEMENE TECHNOLOGIEËN	5
2.1. C#	5
2.2. User Interfaces	5
2.2.1. Windows Forms	5
2.2.2. ASPX	6
2.2.3. Architectuur	6
2.2.3.1. 3-Tier model	6
2.2.3.1.1. DataLaag	6
2.2.3.1.2. BusinessLaag	6
2.2.3.1.3. PresentatieLaag	7
3. PROJECTEN	8
3.1. Aeroscout	8
3.1.1. Analyse	8
3.1.1.1. Klant	8
3.1.1.2. Huidige situatie	8
3.1.1.3. Probleem	8
3.1.1.4. Functionele eisen	9
3.1.1.5. Technische eisen	10
3.1.1.5.1. DataLaag	10
3.1.1.5.2. BusinessLaag	10
3.1.1.5.3. PresentatieLaag	11
3.1.2. Realisatie	12
3.1.2.1. Functionaliteiten	12
3.1.2.2. 3-tier architectuur	12
3.1.2.2.1. DataLaag	12
3.1.2.2.2. PresentatieLaag	13
3.1.2.2.2.1. Asset	13
3.1.2.2.2.2. Group	13
3.1.2.3. Uitdagingen	13
3.1.2.4. Resultaat	14
3.2. Poetscontrole	15
3.2.1. Analyse	15
3.2.1.1. Klant	15
3.2.1.2. Huidige situatie	15
3.2.1.3. Probleem	15
3.2.2. Realisatie	16
3.2.2.1. Functionaliteiten	16
3.2.2.2. 3-tier architectuur	16
3.2.2.2.1. DataLaag	16
3.2.2.2.2. PresentatieLaag	19
3.2.2.2.2.1. Configurator	19
3.2.2.2.2.2. WebApp	22

3.2.2.3. <i>Uitdagingen</i>	23
5.1.1.1. <i>Resultaat</i>	23
5.2. DeviceManager	24
5.2.1. Analyse	24
5.2.1.1. <i>Klant</i>	24
5.2.1.2. <i>Huidige situatie</i>	24
5.2.1.3. <i>Probleem</i>	24
5.2.1.4. <i>Functionele eisen</i>	24
5.2.1.5. <i>Technische eisen</i>	25
5.2.1.5.1. <i>Database</i>	25
5.2.1.5.2. <i>Applicatie</i>	25
5.2.2. Realistatie	26
5.2.2.1. <i>DataBase</i>	26
5.2.2.2. <i>Uitdagingen</i>	26
5.2.2.3. <i>Resultaat</i>	27
5.3. Chatbot	28
5.3.1. Analyse	28
5.3.1.1. <i>Klant</i>	28
5.3.1.2. <i>Huidige situatie</i>	28
5.3.1.3. <i>Probleem</i>	28
5.3.1.4. <i>Functionele eisen</i>	28
5.3.1.5. <i>Technische eisen</i>	29
5.3.1.5.1. <i>Chatbot</i>	29
5.3.2. Realisatie	29
5.3.2.1. <i>Uitdagingen</i>	29
5.3.2.2. <i>Besluit</i>	29
6. BESLUIT	30
LITERATUURLIJST	31
6.1.1. Gebruik van artificiële intelligentie	31

1. Inleiding

In dit document wordt de eigenlijke realisatie van mijn stageprojecten toegelicht. Waar in het projectplan de nadruk lag op de voorbereiding, doelstellingen en geplande aanpak, richt dit document zich op de effectieve uitvoering van deze projecten en de concrete resultaten die hieruit voortgekomen zijn. Het vormt dus een logisch vervolg op het projectplan, waarbij de vooropgestelde doelen worden gekoppeld aan de praktische uitwerking ervan.

Tijdens mijn stage binnen de ICT-afdeling van het ziekenhuis heb ik verschillende projecten gerealiseerd die gericht waren op het ondersteunen en optimaliseren van operationele processen. In dit document worden deze projecten verder uitgediept, met aandacht voor hoe de initiële plannen effectief werden omgezet in werkende oplossingen.

De projecten die aan bod komen, situeren zich onder andere binnen het laboratorium, de linnenkamer, het magazijn en de keuken. Ze omvatten onder meer de ontwikkeling van Windows Forms-applicaties, het automatiseren van registraties en gegevensverwerking, en het verbeteren van de efficiëntie van interne workflows. Hierbij werd gebruikgemaakt van technologieën zoals C#, SQL Server en Excel-integraties,

Per project wordt eerst de beginsituatie en het probleem geschetst, waarna de uitgewerkte oplossing en de implementatie worden besproken. Daarnaast wordt ook stilgestaan bij de gebruikte methodes, de samenwerking met mijn stagementor en collega's, en de belangrijkste leerervaringen tijdens het proces.

Op deze manier geeft dit document een duidelijk en gestructureerd beeld van hoe de vooropgestelde plannen uit het projectplan concreet werden gerealiseerd en welke impact deze projecten hadden binnen het ziekenhuis.

2. Algemene Technologieën

2.1. C#

Tijdens mijn stage in het ziekenhuis van Geel heb ik uitsluitend gewerkt met de programmeertaal C#. Deze keuze werd gemaakt op basis van verschillende technische en praktische voordelen die deze programmeertaal biedt binnen een professionele ziekenhuisomgeving. C# is namelijk sterk geïntegreerd binnen het Microsoft-ecosysteem en sluit daardoor goed aan bij de bestaande infrastructuur en softwaretoepassingen die binnen het ziekenhuis gebruikt worden.

Een belangrijk voordeel van C# is de vlotte samenwerking met technologieën zoals Windows Forms, databanken en diverse API-koppelingen. Hierdoor kunnen applicaties efficiënt ontwikkeld worden en eenvoudig geïntegreerd worden met bestaande systemen. Dit is essentieel in een ziekenhuiscontext, waar software betrouwbaar moet communiceren met andere toepassingen en gegevenssystemen.

Daarnaast biedt C# een hoge mate van compatibiliteit binnen verschillende Windows-omgevingen. In het ziekenhuis werken medewerkers op uiteenlopende toestellen met verschillende schermgroottes en configuraties. Het is daarom cruciaal dat applicaties stabiel functioneren op alle werkstations zonder bijkomende aanpassingen of configuraties.

Verder speelt ook de objectgeoriënteerde structuur van C# een belangrijke rol. Door gebruik te maken van klassen, methodes en herbruikbare componenten kan code efficiënt georganiseerd en onderhouden worden. Dit verhoogt niet alleen de leesbaarheid van de applicaties, maar maakt het ook eenvoudiger om bestaande functionaliteiten uit te breiden of opnieuw te gebruiken in toekomstige projecten.

Samengevat biedt C# een sterke combinatie van gebruiksvriendelijkheid, flexibiliteit en betrouwbaarheid. Deze eigenschappen maken de programmeertaal bijzonder geschikt voor het ontwikkelen van onderhoudsvriendelijke en performante software binnen een dynamische en veeleisende ziekenhuisomgeving.

2.2. User Interfaces

2.2.1. Windows Forms

Binnen het ziekenhuis van Geel wordt voornamelijk gebruikgemaakt van Windows Forms voor de ontwikkeling van desktopapplicaties. Deze technologie biedt verschillende voordelen die goed aansluiten bij de dagelijkse werking van een ziekenhuisomgeving. Een van de belangrijkste voordelen is de eenvoudige distributie van applicaties binnen het interne netwerk van het ziekenhuis. Hierdoor kunnen medewerkers snel en efficiënt toegang krijgen tot de nodige softwaretoepassingen, zonder complexe installaties of configuraties.

Daarnaast is Windows Forms een schaalbare technologie. Bestaande applicaties kunnen relatief eenvoudig aangepast of uitgebreid worden wanneer nieuwe functionaliteiten nodig zijn. Dit is bijzonder belangrijk in een omgeving waar processen regelmatig evolueren en software flexibel moet kunnen inspelen op veranderende noden. Dankzij deze schaalbaarheid hoeft een applicatie niet volledig opnieuw ontwikkeld te worden bij kleine of middelgrote wijzigingen, wat zowel tijd als ontwikkelingskosten bespaart.

De keuze om met Windows Forms te werken werd door het ziekenhuis zelf gemaakt en maakte reeds deel uit van de bestaande IT-infrastructuur. Tijdens mijn stage heb ik echter kunnen vaststellen dat deze technologie nog steeds goed aansluit bij de operationele behoeften van het personeel. Vooral voor interne administratieve toepassingen biedt Windows Forms een stabiele, betrouwbare en gebruiksvriendelijke oplossing.

2.2.2. ASPX

Voor bepaalde toepassingen binnen het ziekenhuis is het belangrijk dat medewerkers toegang hebben tot applicaties via verschillende apparaten en locaties, en niet uitsluitend via de standaard werkstations binnen het netwerk. In dergelijke situaties wordt gebruikgemaakt van ASPX-technologie. ASPX is een webtechnologie binnen het Microsoft-platform die HTML combineert met C# om dynamische webapplicaties te ontwikkelen.

Dankzij ASPX kunnen applicaties via een webbrowser geraadpleegd worden, waardoor gebruikers meer flexibiliteit hebben in de manier waarop zij toegang krijgen tot informatie en functionaliteiten. Dit biedt voordelen in situaties waar mobiliteit of externe toegang belangrijk is. Medewerkers kunnen hierdoor eenvoudiger werken op verschillende toestellen zonder dat specifieke software lokaal geïnstalleerd moet worden.

Hoewel ASPX verschillende mogelijkheden biedt op vlak van toegankelijkheid en platformonafhankelijkheid, wordt deze technologie binnen het ziekenhuis minder frequent gebruikt dan Windows Forms. De meeste interne toepassingen draaien namelijk lokaal op vaste werkstations binnen het ziekenhuisnetwerk. Hierdoor blijft Windows Forms voor veel administratieve en operationele processen de meest efficiënte oplossing. ASPX wordt voornamelijk ingezet wanneer een webgebaseerde aanpak een duidelijke meerwaarde biedt voor de gebruikers of de werking van de applicatie.

2.2.3. Architectuur

2.2.3.1. 3-Tier model

Het 3-tier model is een veelgebruikte softwarearchitectuur waarbij een systeem wordt opgesplitst in drie afzonderlijke lagen: de presentatielaag, de applicatielaag (businesslaag) en de datalaag. Deze duidelijke scheiding van verantwoordelijkheden zorgt ervoor dat het systeem overzichtelijker wordt opgebouwd en eenvoudiger te onderhouden is. Bovendien verhoogt dit de schaalbaarheid van de applicatie, aangezien elke laag onafhankelijk kan worden aangepast of uitgebreid zonder directe impact op de andere lagen.

2.2.3.1.1. DataLaag

De datalaag is verantwoordelijk voor het beheren van alle gegevens binnen het systeem. In deze laag worden data opgeslagen, opgehaald en bijgewerkt via databases of andere opslagstructuren. De communicatie met deze laag gebeurt uitsluitend via de applicatielaag, die de juiste bedrijfsregels toepast voordat gegevens worden verwerkt. Door deze strikte scheiding blijft de data consistent en goed beveiligd, wat essentieel is in omgevingen waar betrouwbaarheid en integriteit van informatie cruciaal zijn. Daarnaast draagt deze aanpak bij aan een betere onderhoudbaarheid en schaalbaarheid van het systeem, omdat de opslagstructuur onafhankelijk kan evolueren van de rest van de applicatie.

2.2.3.1.2. BusinessLaag

De businesslaag, ook wel applicatielaag genoemd, vormt het centrale onderdeel van het 3-tier model. In deze laag wordt de kernlogica van de applicatie uitgevoerd. Ze ontvangt gegevens vanuit de presentatielaag, verwerkt deze volgens vooraf gedefinieerde bedrijfsregels en stuurt de resultaten vervolgens door naar de datalaag of terug naar de gebruikersinterface. Belangrijke taken zoals validatie van gegevens, het uitvoeren van berekeningen en het coördineren van processen vinden hier plaats. Dankzij deze scheiding blijft de applicatie flexibel en goed gestructureerd, waardoor onderhoud en toekomstige uitbreidingen eenvoudiger te realiseren zijn.

2.2.3.1.3. PresentatieLaag

De presentatielaag vormt de schakel tussen de gebruiker en het achterliggende systeem. Deze laag is verantwoordelijk voor alle interacties met de gebruiker, zoals het invoeren van gegevens via formulieren, knoppen en andere interface-elementen, evenals het weergeven van resultaten. De presentatielaag stuurt ingevoerde informatie door naar de businesslaag, waar deze verder verwerkt wordt, en ontvangt vervolgens de verwerkte gegevens om ze op een duidelijke en gebruiksvriendelijke manier te presenteren. Door deze laag strikt gescheiden te houden van de logica en de dataverwerking kan de gebruikersinterface onafhankelijk aangepast of verbeterd worden, zonder dat dit invloed heeft op de onderliggende werking van de applicatie.

3. Projecten

3.1. Aeroscout

3.1.1. Analyse

3.1.1.1. Klant

De klant voor deze applicatie is voornamelijk de keukenafdeling van het ziekenhuis. Deze afdeling ondervond problemen bij het opvolgen en beheren van de voorraad in de verschillende opslagkasten. Door het gebrek aan een efficiënt overzicht werd regelmatig vastgesteld dat producten te laat werden gecontroleerd of over de houdbaarheidsdatum gingen, wat leidde tot voedselverspilling. De keuken is verantwoordelijk voor de bereiding van alle maaltijden binnen het ziekenhuis, zowel voor patiënten als voor medewerkers. Hierdoor is een correct en actueel voorraadbeheer essentieel om de continuïteit van de voedselvoorziening te garanderen en verspilling tot een minimum te beperken.

3.1.1.2. Huidige situatie

Op dit moment bestaat de applicatie reeds en wordt deze binnen het volledige ziekenhuis gebruikt om nauwkeurige metingen en uitlezingen te voorzien voor elke koelkast en diepvries. Deze continue monitoring is van cruciaal belang, aangezien zelfs korte periodes van te hoge of te lage temperaturen kunnen leiden tot het verlies van zowel voedingsmiddelen als, in kritieke gevallen, medische materialen zoals organen die onder strikt gecontroleerde omstandigheden bewaard moeten worden.

Naast de basisfunctionaliteit voor temperatuurmeting is de applicatie gekoppeld aan meerdere bijkomende modules. In totaal zijn er zeven gerelateerde projecten ontwikkeld die de gegevens op verschillende manieren visualiseren en verwerken. Deze projecten maken het mogelijk om de informatie op een gebruiksvriendelijke manier te raadplegen, afhankelijk van de noden van de gebruiker. Bovendien zorgen ze ervoor dat er automatische meldingen gegenereerd kunnen worden wanneer er afwijkingen of problemen worden vastgesteld, zodat er snel kan worden ingegrepen.

3.1.1.3. Probleem

Doordat elke koelkast binnen het ziekenhuis beschikt over een specifieke codenaam, zoals bijvoorbeeld APT-KAST1, kan het voor nieuwe medewerkers moeilijk zijn om snel de juiste koelkast te identificeren en correct te interpreteren welke locatie bij welke naam hoort. Deze coderingen zijn weliswaar functioneel voor het systeem, maar minder intuïtief voor gebruikers die nog niet vertrouwd zijn met de interne structuur. Daarnaast blijkt in de praktijk dat de pop-upmeldingen die waarschuwen voor afwijkingen of problemen niet altijd de gewenste effectiviteit hebben. Deze meldingen worden soms over het hoofd gezien of bewust genegeerd, waardoor kritieke informatie niet tijdig wordt opgevolgd. Een bijkomend probleem is dat de verantwoordelijke medewerker niet altijd fysiek aanwezig is aan de computer op het moment dat een melding verschijnt. Hierdoor kan de waarschuwing gemist worden, wat het risico op vertraagde interventies vergroot en mogelijk gevolgen heeft voor de bewaring van temperatuurgevoelige goederen.

3.1.1.4. Functionele eisen

De functionele eisen beschrijven wat de applicatie na de aanpassingen moet kunnen en welke extra functionaliteiten worden verwacht ten opzichte van de bestaande situatie. Met andere woorden bepalen deze eisen welke taken en processen de applicatie moet ondersteunen vanuit het perspectief van de eindgebruiker.

In dit project ligt de nadruk vooral op de functionele werking van het systeem, los van de technische implementatie. Het gaat dus om wat de applicatie moet doen, niet hoe dit technisch wordt gerealiseerd. De functionele eisen definiëren onder andere welke informatie beschikbaar moet zijn, hoe gebruikers met de applicatie moeten kunnen omgaan en welke acties het systeem moet ondersteunen om de dagelijkse werking binnen de keuken en het ziekenhuis efficiënter en betrouwbaarder te maken.

Voor dit project zijn is op voorhand afgesproken en aan mij gegeven de Functionele eisen zijn de volgende use cases

USE CASE 1

Naam: Alias kasten

Actors: Medewerker

Functionaliteit: als medewerker kan ik snel zien over welke kast er wordt gesproken

Voorwaarden: De alias moet op voorhand worden doorgegeven aan de ICT-dienst & er moet een fout zijn met de temperatuur van een koelkast

Normale flow:

Systeem laat een popup zien op de computer in de keuken dat er iets mis is met de computer en gebruikt hiervoor zowel de kastnaam als het alias. De medewerker kan makkelijk zien welke kast een foutmelding geeft en dit oplossen

Alternative flow:

/

USE CASE 2

Naam: Telefoonnummer Kast

Actors: Verantwoordelijke voor de kast

Functionaliteit: als kast Verantwoordelijke krijg ik een sms als er iets mis is met de koelkasten waar hij/zij verantwoordelijk voor is

Voorwaarden: het telefoonnummer is vooraf doorgegeven aan de ICT dienst & er is een probleem met een koelkast

Normale flow: Het systeem logt een probleem met een koelkast naar de database en stuurt een sms naar de verantwoordelijke. De verantwoordelijke krijgt een sms aan en kan op een juiste manier verder gaan

Alternatieve flow:

De verantwoordelijke heeft geen signaal: de sms komt later aan bij het diensthoofd

USE CASE 3

Naam: Telefoonnummer Diensten

Actors: Diensthoofd

Functionaliteit: als Diensthoofd krijg ik een sms als er iets mis is met de koelkasten in zijn dienst

Voorwaarden: het telefoonnummer is vooraf doorgegeven aan de ICT dienst & er is een probleem met een koelkast

Normale flow: Het systeem logt een probleem met een koelkast naar de database en stuurt een sms naar het diensthoofd. Het diensthoofd krijgt een sms aan en kan op een juiste manier verder gaan

Alternatieve flow:

Het diensthoofd heeft geen signaal: de sms komt later aan bij het diensthoofd

3.1.1.5. Technische eisen

Het document met de technische eisen beschrijft op welke manier de applicatie technisch wordt opgebouwd en hoe de verschillende onderdelen met elkaar zullen samenwerken. In tegenstelling tot de functionele eisen, die bepalen wat de applicatie moet doen vanuit het perspectief van de gebruiker, focussen de technische eisen op de concrete technische uitwerking van deze functionaliteiten.

Binnen dit onderdeel wordt vastgelegd welke architectuur wordt gebruikt, hoe de verschillende lagen van het systeem met elkaar communiceren en welke technologieën en frameworks worden toegepast. Daarnaast worden ook aspecten zoals databankstructuur, performantie, onderhoudbaarheid en schaalbaarheid in rekening gebracht.

De technische eisen vormen dus de vertaling van de functionele vereisten naar een technisch realiseerbare oplossing. Ze zorgen ervoor dat de applicatie op een consistente, efficiënte en toekomstgerichte manier kan worden ontwikkeld en onderhouden.

3.1.1.5.1. DataLaag:

Sinds deze applicatie al werkte voor dat ik eraan begon en ik enkel velden moet toevoegen

Assets (Koelkasten):

In de tabel assets heb ik de volgende velden toegevoegd:

- AssetAlias
 - Dit is toegevoegd om een kast een bijnaam te geven.
- Assettelefoon
 - Dit is toegevoegd om een telefoonnummer aan een kast te hangen en deze dan te benoemen als “kastverantwoordelijke”

Groups (diensten):

In de tabel groups heb ik de volgende velden toegevoegd:

- GsmNumber
 - Dit is om telefoonnummers te hangen aan een dienst en later de diensthoofden te verwittigen als er een probleem is met een koelkast

3.1.1.5.2. BusinessLaag

De businesslaag vormt de kern van de applicatie. In deze laag bevindt zich de volledige logica die de werking van het systeem ondersteunt. Hier worden de belangrijkste domeinobjecten gedefinieerd, elk met hun eigen eigenschappen en methodes om de gegevens correct te beheren en te verwerken. Door deze logica centraal in de businesslaag te plaatsen, blijft de applicatie consistent, overzichtelijk en beter onderhoudbaar. Bovendien zorgt deze scheiding ervoor dat de complexiteit van de applicatie beter beheersbaar blijft en wijzigingen eenvoudiger kunnen worden doorgevoerd zonder andere lagen te beïnvloeden.

Binnen deze laag worden onder andere de objecten Assets en Groups gebruikt.

Het object Assets stelt de koelkasten binnen het ziekenhuis voor en dient om deze centraal te beheren. Voor elke koelkast wordt een unieke codenaam bijgehouden, bijvoorbeeld MED-710. Daarnaast wordt ook een alias opgeslagen. Dit veld werd toegevoegd om de gebruiksvriendelijkheid te verhogen: diensthoofden kunnen een herkenbare naam doorgeven die het voor nieuwe medewerkers eenvoudiger maakt om de juiste koelkast terug te vinden binnen het systeem.

Verder bevat het object ook een veld voor een telefoonnummer, een extra attribuut dat werd toegevoegd zodat diensthoofden een contactnummer kunnen koppelen aan een specifieke koelkast. Op die manier kan bij een probleem automatisch een melding of verwittiging naar dit nummer worden gestuurd. Daarnaast bevat het object nog een attribuut om aan te geven of een koelkast actief is, zodat inactieve of buiten gebruik zijnde toestellen gefilterd kunnen worden. Tot slot is er nog een veld dat de nodige actie bij een storing of afwijking bepaalt, zodat het systeem correct kan reageren afhankelijk van de situatie.

Het object Groups wordt gebruikt om de verschillende diensten binnen het ziekenhuis te beheren. Voor elke dienst wordt de naam geregistreerd, samen met het gsm-nummer van het diensthoofd. Dit veld werd toegevoegd zodat het systeem in de toekomst ook sms-meldingen kan versturen bij problemen met een koelkast binnen de betreffende dienst. Daarnaast wordt ook het e-mailadres van het diensthoofd bijgehouden, zodat er bijkomende communicatiekanalen beschikbaar zijn voor belangrijke meldingen en opvolging.

3.1.1.5.3. PresentatieLaag

De presentatielaag, ook wel de visuele laag genoemd, vormt de interface tussen de gebruiker en de applicatie. In deze laag wordt alle interactie met de gebruiker afgehandeld via de user interface. Dit omvat onder andere schermen, formulieren, knoppen en andere visuele componenten waarmee gegevens kunnen worden ingevoerd en resultaten kunnen worden weergegeven.

Voor deze applicatie heb ik uitsluitend gewerkt met Windows Forms binnen deze presentatielaag. Daarbij heb ik enkel bestaande forms aangepast en uitgebreid, in plaats van volledig nieuwe interfaces te ontwikkelen. Dit betekent dat de focus voornamelijk lag op het verbeteren en optimaliseren van de bestaande gebruikersschermen, zodat deze beter aansluiten bij de noden van de eindgebruikers.

Daarnaast heb ik voornamelijk gewerkt binnen de configurator. Dit onderdeel laat toe om instellingen en configuraties van de applicatie te beheren, zonder dat de onderliggende logica aangepast moet worden. Op die manier kon ik gerichte aanpassingen doorvoeren in de presentatie van de gegevens, terwijl de business- en data laag onaangeroerd bleven.

CONFIGURATIEDEEL - ASSET

In dit onderdeel bespreek ik alles wat ik in de presentatielaag heb gedaan voor Asset. Spijtig genoeg kan ik geen foto's meegeven sinds hier soms gevoelige informatie blijkt.

- Toevoeging aan configuratieform
 - In de configuratieform voor assets zowel aan te maken als aan te passen heb ik mijn nieuwe velden toegevoegd
- Plan van aanpak sms
 - Voor de smsen te sturen heb ik een plan van aanpak gemaakt hoe ik dat zou implementeren. Later is deze gebruikt om het te implanteren

CONFIGURATIEDEEL - GROUP

In dit onderdeel bespreek ik alles wat ik in de presentatielaag heb gedaan voor Group. Spijtig genoeg kan ik geen foto's meegeven sinds hier soms gevoelige informatie blijkt.

- Toevoeging aan configuratieform
 - In de configuratieform voor Group zowel aan te maken als aan te passen heb ik mijn nieuwe velden toegevoegd en een optie gemaakt om ernaartoe te gaan

3.1.2. Realisatie

3.1.2.1. Functionaliteiten

- Alias kunnen doorgeven
- Alias kunnen bekijken
- Telefoonnummer kunnen doorgeven per koelkast
- Meerdere telefoonnummer kunnen doorgeven voor diensten

3.1.2.2. 3-tier architectuur

3.1.2.2.1. DataLaag

dbo.Assets

	Column Name	Data Type	Allow Nulls
	Id	int	☐
	AssetName	nvarchar(50)	☐
	AssetAlias	nvarchar(50)	☒
	AssetTelefoon	nvarchar(50)	☒
	Active	bit	☐
	ActionImposed	nvarchar(MAX)	☒
			☐

Figuur 1: Database tabel assets

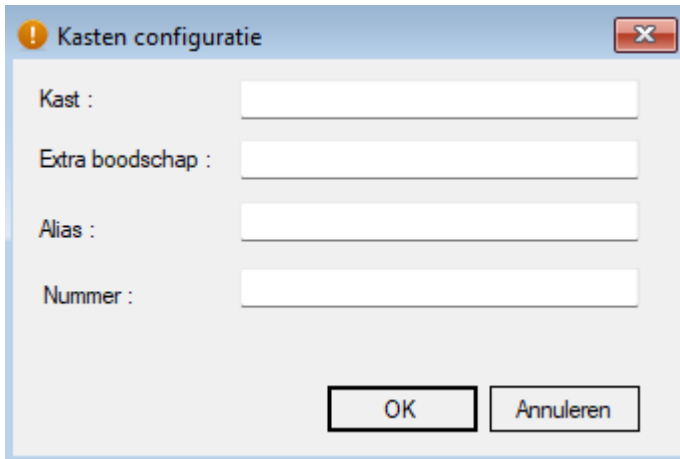
dbo.Groups

	Column Name	Data Type	Allow Nulls
	Id	int	☐
	GroupName	nvarchar(50)	☐
	GroupEmail	nvarchar(250)	☒
	GsmNumber	nvarchar(50)	☒
			☐

Figuur 2: Database tabel groups

3.1.2.2.2. PresentatieLaag

3.1.2.2.2.1. Asset



! Kasten configuratie

Kast :

Extra boodschap :

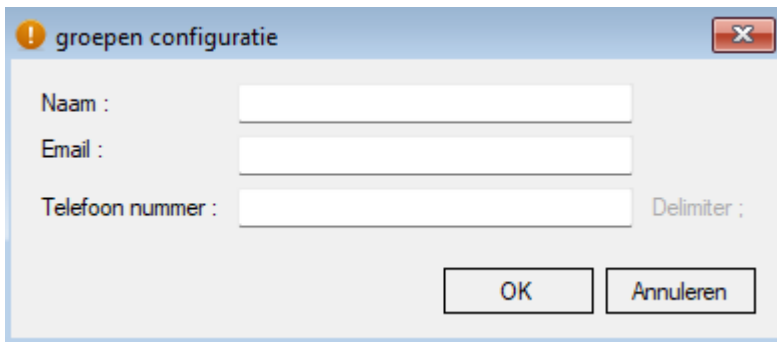
Alias :

Nummer :

OK Annuleren

Figuur 3: Asset presentatie-laag 1

3.1.2.2.2.2. Group



! groepen configuratie

Naam :

Email :

Telefoon nummer : Delimiter ;

OK Annuleren

Figuur 4: Group presentatie-laag 2

3.1.2.3. Uitdagingen

Tijdens de uitvoering van dit project heb ik over het algemeen weinig technische uitdagingen ervaren. De ontwikkeling verliep relatief vlot, voornamelijk omdat de bestaande structuur en architectuur van de applicatie al goed uitgewerkt waren en ik hierop kon verder bouwen. Hierdoor lag de focus vooral op het aanpassen en uitbreiden van bestaande functionaliteiten in plaats van het volledig opbouwen van nieuwe onderdelen.

De grootste uitdaging binnen dit project was het correct leren en begrijpen van Windows Forms. In het begin was het niet altijd duidelijk hoe de verschillende componenten, events en bindings precies met elkaar samenwerken. Het vergt enige tijd om te begrijpen hoe user interfaces efficiënt opgebouwd worden binnen dit framework en hoe de interactie tussen de interface en de achterliggende logica het best georganiseerd kan worden.

Na verloop van tijd werd dit echter duidelijker door ervaring op te doen en bestaande forms aan te passen. Hierdoor kon ik stap voor stap beter inschatten hoe Windows Forms optimaal gebruikt kan worden binnen de context van deze applicatie.

3.1.2.4. Resultaat

Het resultaat van dit project sluit aan bij de vooraf gestelde doelstellingen en voldoet aan de verwachtingen. Na de implementatie is het mogelijk om een alias toe te voegen aan een koelkast, waardoor deze op een meer herkenbare en gebruiksvriendelijke manier kan worden weergegeven binnen de applicatie. Deze alias wordt vervolgens correct weergegeven op de relevante plaatsen in het systeem.

Daarnaast is het ook mogelijk geworden om gsm-nummers toe te voegen aan zowel koelkasten als diensten. Hierdoor kunnen contactgegevens rechtstreeks gekoppeld worden aan de juiste entiteiten binnen het systeem, wat de communicatie en opvolging van meldingen aanzienlijk verbetert. Deze uitbreiding draagt bij aan een efficiënter beheer en een betere respons bij eventuele problemen of waarschuwingen binnen het ziekenhuis.

3.2. Poetscontrole

3.2.1. Analyse

3.2.1.1. Klant

Voor deze applicatie is de primaire klant het hoofd van de facultaire dienst, die verantwoordelijk is voor het opvolgen en organiseren van de schoonmaakactiviteiten binnen het ziekenhuis. De applicatie is ontwikkeld om deze gebruiker te ondersteunen bij het eenvoudig controleren en opvolgen van de werkzaamheden van de poetsploegen.

Daarnaast biedt het systeem de mogelijkheid om op langere termijn analyses te maken van de schoonmaakprestaties. Hierdoor kan het diensthoofd beter inschatten welke zones in het ziekenhuis extra aandacht vereisen of waar eventuele tekortkomingen in de schoonmaakplanning optreden. Aangezien hygiëne binnen een ziekenhuisomgeving van essentieel belang is, speelt een correcte opvolging van schoonmaakactiviteiten een cruciale rol in het waarborgen van een veilige en propere werkomgeving voor zowel patiënten als personeel.

3.2.1.2. Huidige situatie

Op dit moment worden binnen het ziekenhuis reeds poetscontroles uitgevoerd die tot in groot detail worden nagekeken. Hierbij worden onder andere specifieke markeringen gebruikt, zoals stippen die enkel zichtbaar zijn onder speciaal licht, om na te gaan of bepaalde oppervlakken correct gereinigd werden. Deze aanpak maakt het mogelijk om de kwaliteit van de schoonmaak zeer nauwkeurig te evalueren.

Echter worden deze controles momenteel ondersteund door verschillende applicaties en systemen.

Hierdoor ontstaat er in de praktijk soms verwarring bij de gebruikers, omdat informatie verspreid zit over meerdere platforms en niet altijd op een uniforme manier wordt weergegeven. Dit kan leiden tot een minder efficiënt opvolgproces en verhoogt de kans op fouten of onduidelijkheden bij de rapportering en opvolging van de schoonmaakresultaten.

3.2.1.3. Probleem

Doordat de poetscontroles momenteel in verschillende applicaties worden ingevoerd en beheerd, ontstaat er een versnippering van de beschikbare informatie. Dit maakt het in de praktijk soms moeilijk om snel en efficiënt de juiste gegevens terug te vinden of een volledig overzicht van de uitgevoerde controles te bekomen.

In sommige gevallen kan deze gefragmenteerde aanpak er bovendien toe leiden dat gegevens onvolledig worden geregistreerd of moeilijk terug te vinden zijn. In extreme situaties kan dit zelfs resulteren in het verlies van belangrijke informatie, wat de betrouwbaarheid van de opvolging en rapportering van de poetscontroles negatief beïnvloedt.

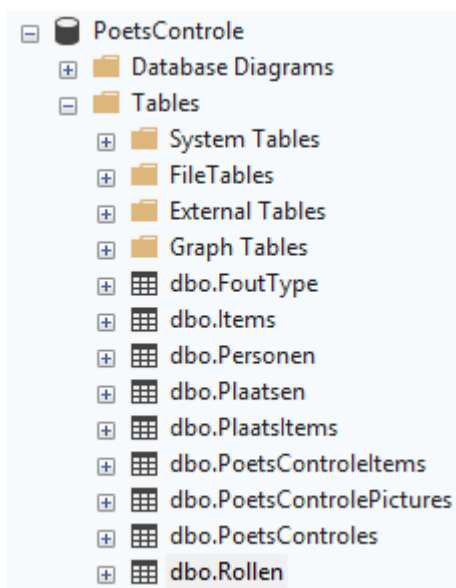
3.2.2. Realisatie

3.2.2.1. Functionaliteiten:

- Medewerkers kunnen poetsers nakijken en aanduiden per item wat er mis is met meerdere foto's
- Medewerkers kunnen terugkijken op poetscontroles van een dag en een handtekening van een poetser ophalen om later te weten dat de poets weet wat er fout is
- Als er extra problemen zijn kan het hoofd van facultaire diensten worden gemaild met de nodige informatie
- Medewerkers kunnen items afwezig zetten, weghalen of bij in een kamer zetten
- Het hoofd van facultaire diensten kan elk object aanmaken en de nodige objecten verbinden
- Het hoofd van facultaire diensten kan een analyse bekijken en exporteren van de meest gemaakt fouten, de plaatsen met de meeste fouten, de medewerkers met de meeste fouten en de items met de meeste items voor bijscholing
- Het hoofd van facultaire diensten kan terug gaan kijken op poetscontroles en de foto's bekijken en verwijderen

3.2.2.2. 3-tier architectuur

3.2.2.2.1. DataLaag



Figuur 53: SQL database structuur

	Column Name	Data Type	Allow Nulls
🔑	ID	int	☐
	Naam	varchar(50)	☒
	Actief	bit	☒
▶			☐

Figuur 6: Database tabel fouttypes

	ID	int	☐
	Naam	varchar(50)	☒
	Opmerking	varchar(150)	☒
	Actief	bit	☒
			☐

Figuur 74:Database tabel items

	ID	int	☐
	Naam	varchar(50)	☒
	Rollid	int	☒
	Actief	bit	☒
			☐

Figuur 85:Database tabel personen

	ID	int	☐
	KamerNummer	varchar(50)	☒
	UltimoNummer	varchar(50)	☒
	OppRuimte	int	☒
	OppRamen	int	☒
	Opmerkingen	varchar(150)	☒
	Actief	bit	☒
			☐

Figuur 96:Database tabel plaatsen

	ID	int	☐
	PlaatsId	int	☒
	ItemId	int	☒
	Opmerking	varchar(50)	☒
			☐

Figuur 107:Database tabel plaatsitem

	ID	int	☐
	PoetsControlesId	int	☒
	ItemId	int	☒
	FoutId	int	☒
	Opmerking	varchar(150)	☒
	IsGoedGekeurd	bit	☒
			☐

Figuur 11: Database tabel poetscontroleitems

	ID	int	☐
	PoetsControleItemID	int	☒
	Path	varchar(250)	☒
			☐

Figuur 12: Database tabel poetscontrolepictures

	ID	int	☐
	PlaatsID	int	☒
	PersoneelID	int	☒
	UitvoerderID	int	☒
	Datum	datetime	☒
	PoetserHandTekening	varbinary(MAX)	☒
			☐

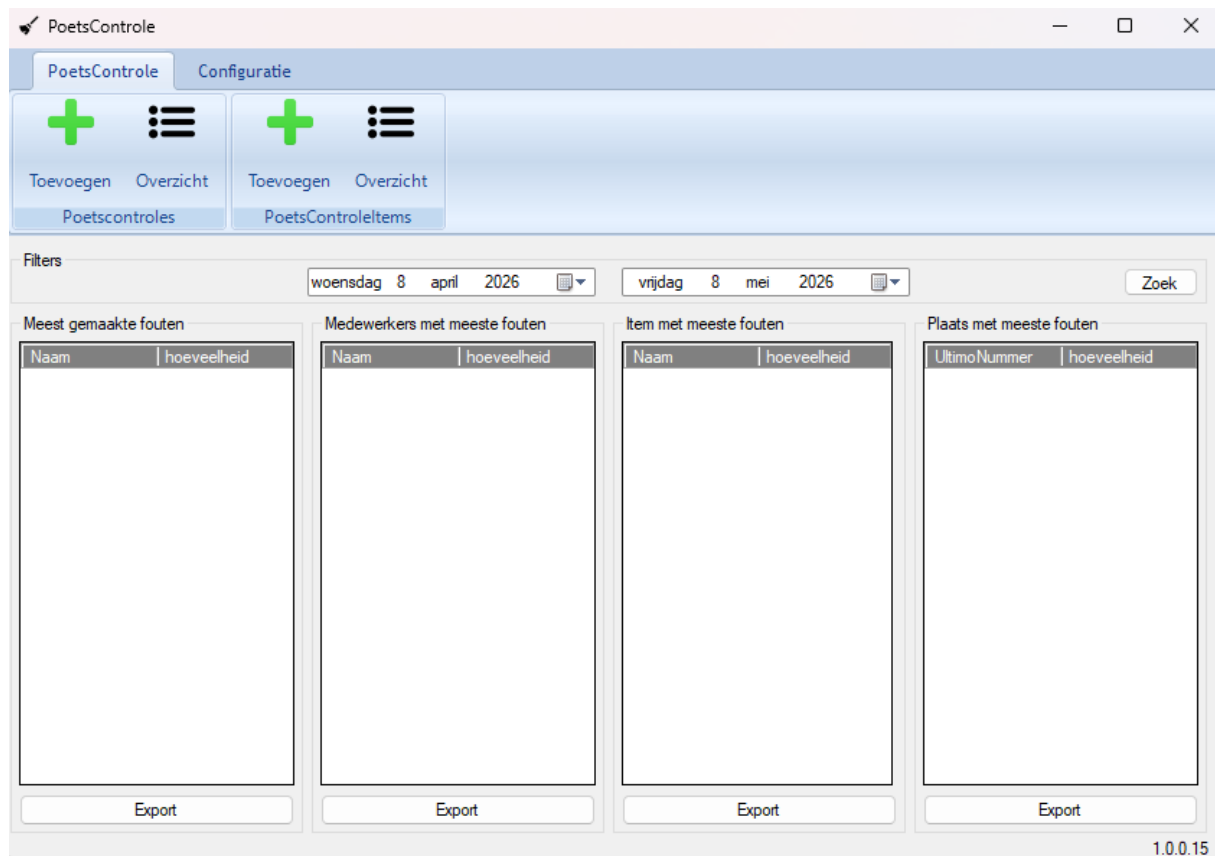
Figuur 13: Database tabel poetscontroles

	ID	int	☐
	Naam	varchar(50)	☒
	Actief	bit	☒
			☐

Figuur 14: Database tabel rollen

3.2.2.2.2. PresentatieLaag

3.2.2.2.2.1. Configurator



Figuur 15: Presentatie laag formmain

Overzicht Plaatsen

×

Filter

Kamer Naam :

Ultimo Naam :

Oppervlakte Ruimte :

0

▲▼

m²

Oppervlakte Ramen :

0

▲▼

m²

Opmerking :

Actief :

▼

Overzicht

Kamer Nummer	Ultimo Nummer	Opp Ruimte	Opp Ramen	Opmerkingen	Actief

Nieuw

Kopieren

Open

Verwijderen

Sluiten

Figuur 16: Presentatie laag plaatsen overzicht

Plaats : Nieuw

Details

Kamer Nummer :

Ultimo Nummer :

Oppervlakte Ruimte : m²

Oppervlakte Ramen : m²

Items

Itemnaam

Opmerkingen :

☒ Actief

Figuur 17: Presentatie laag plaatsen detail

3.2.2.2.2. WebApp

Plaatsen

Fouten

Mail Fac.

1.0.0.15

Filter

☐ Blok
 ☐ Verdieping
 Nummer

test

Search

kies een kamer :

Kamer	Ultimo nummer	Opp Ruimte	Opp Ramen
test	test	1	1

Poetser:

Selecteer een poetser

items van test

Naam	Aanwezig	Fout	Opmerking	Commentaar	Verwijderen
Afvalkarren 1	☒	Meld			
Afvoerputje 1	☒	Meld			
Alcoholgelhouder 1	☒	Meld			

Controle Beëindigen

Figuur 18: Presentatie laag webapp default

testmedewerker

Plaatsen

Fouten

1.0.0.15

08/05/2026

Search

Kamer	Ultimo nummer	Poetser	Datum	Status
test	test	testpoetser	8/05/2026 9:51:35	niet geaccepteerd

Nagekeken Items

Item	Fout	Foto	Opmerking
Afvalkarren 1	testfouttypes	Foto's N bekijken	
Afvoerputje 1			
Alcoholgelhouder 1			

Save

Figuur 19: presentatie laag webapp fouten

3.2.2.3. Uitdagingen

Binnen dit project heb ik verschillende uitdagingen ervaren, voornamelijk door de omvang en complexiteit van de applicatie. Hieronder worden de belangrijkste moeilijkheden toegelicht.

4. Windows Forms

- **Plaatsen en beheren van items in een overzichtsscherm**

In het overzichtsscherm moest een manier ontwikkeld worden om items tijdelijk bij te houden zonder deze onmiddellijk op te slaan in de databank. Hiervoor heb ik meerdere ontwerpbenaderingen uitgewerkt en getest, waarna ik uiteindelijk de meest geschikte oplossing heb gekozen die zowel performant als overzichtelijk was.

- **Sorteren op datum bij wijziging van datumvelden**

Een onverwacht probleem deed zich voor bij het aanpassen van datumvelden in combinatie met debugging (breakpoints). In bepaalde situaties leidde dit tot systeeminstabiliteit, waarbij mijn computer vastliep. Na onderzoek bleek dit vermoedelijk te wijten te zijn aan een probleem in de combinatie van debugging en Windows Forms. Dit heb ik opgelost door de werkwijze aan te passen en de problematische interactie te vermijden.

5. Webapplicatie

- **Beheer van items en achterliggende lijsten**

Binnen de webapplicatie gaat de status van de pagina verloren bij elke interactie, waardoor het noodzakelijk was om alle dynamische elementen zoals checkboxes en tekstvelden op te slaan in de ViewState. Dit vereiste dat bij elke actie de volledige toestand van de pagina opnieuw werd opgebouwd, wat het beheer van items complex en foutgevoelig maakte.

- **Opslag van poetscontroles**

Het opslaan van poetscontroles bleek uitdagend door de vele mogelijke acties binnen één controle, zoals het toevoegen en verwijderen van items, het valideren van controles en het toevoegen van foto's. Het correct bepalen van de volgorde waarin gegevens moesten worden opgeslagen, was hierbij essentieel om dataconsistentie te garanderen.

- **Permissies en beheer van foto's**

Voor het verwerken van foto's was het niet toegestaan om deze permanent op te slaan buiten de context van een opgeslagen poetscontrole. Daarom werden de afbeeldingen eerst in een tijdelijke map opgeslagen met tijdelijke bestandsnamen. Aanvankelijk werden unieke namen gegenereerd op basis van timestamps, maar door de hoge verwerkingssnelheid leidde dit tot conflicten. Uiteindelijk was het noodzakelijk om extra precisie toe te voegen (meer decimalen in de timestamp) om unieke bestandsnamen te garanderen. Daarnaast zorgden ontbrekende toegangsrechten in het begin voor bijkomende problemen, waardoor de correcte opslagstructuur pas na verschillende aanpassingen werd bereikt.

5.1.1.1. Resultaat

- Het resultaat van deze opdracht kan beschouwd worden als het meest volledige en uitgewerkte project binnen mijn stageperiode. De applicatie is op een gestructureerde manier afgewerkt en bevat alle vooropgestelde functionaliteiten die in de analysefase werden bepaald. Deze functionaliteiten zijn bovendien succesvol geïmplementeerd en werken correct binnen de applicatie.
- Daarnaast werd er aandacht besteed aan stabiliteit, gebruiksvriendelijkheid en de correcte verwerking van gegevens, wat ervoor zorgt dat de applicatie betrouwbaar kan worden ingezet in de praktijk. Door de combinatie van een goede architectuur en een correcte implementatie sluit het eindresultaat nauw aan bij de oorspronkelijke doelstellingen van het project.

5.2. DeviceManager

5.2.1. Analyse

5.2.1.1. Klant

Voor deze applicatie is de primaire klant de ICT-dienst van het ziekenhuis. Deze dienst is verantwoordelijk voor het beheer, onderhoud en de verdere ontwikkeling van de interne IT-systemen. De applicatie werd ontwikkeld als antwoord op de noodzaak om bestaande, verouderde systemen te vernieuwen en beter af te stemmen op de huidige technologische standaarden en gebruikersbehoeften.

De ICT-dienst fungeert hierbij zowel als eindgebruiker als beheerder van het systeem. Hierdoor ligt de focus niet alleen op gebruiksvriendelijkheid, maar ook op onderhoudbaarheid, uitbreidbaarheid en integratie binnen de bestaande IT-infrastructuur.

5.2.1.2. Huidige situatie

Op dit moment bestaat de Device Manager-applicatie reeds en wordt deze binnen het ziekenhuis gebruikt om verschillende devices te beheren, zoals pc's, printers, switches en andere netwerkcomponenten. De huidige implementatie is grotendeels opgebouwd in Visual Basic en maakt deel uit van een bestaand systeem dat door de ICT-dienst wordt onderhouden.

Naast de applicatie zelf wordt er ook gebruikgemaakt van scripts om gegevens op te halen en te verwerken. Deze scripts zijn momenteel gekoppeld aan Excel-bestanden, die dienen als bron voor het verzamelen en structureren van informatie over de verschillende devices. Hoewel deze aanpak functioneel is, zorgt ze ervoor dat het beheer en de data-uitwisseling sterk afhankelijk zijn van manuele of minder geïntegreerde processen.

5.2.1.3. Probleem

Doordat de applicatie en de scripts momenteel los van elkaar functioneren, ontstaat er een gefragmenteerde werkwijze binnen het beheerproces. De scripts zijn bovendien niet door alle medewerkers toegankelijk of bruikbaar, wat het voor bepaalde gebruikers moeilijk maakt om snel en efficiënt informatie op te vragen, bijvoorbeeld over een specifieke switchpoort.

Daarnaast wordt er momenteel geen structurele historiek of persistente data bijgehouden over de configuratie en status van switchpoorten. Hierdoor ontbreekt een centraal overzicht van wijzigingen en huidige instellingen, wat het beheer minder transparant maakt en het moeilijker kan maken om problemen te analyseren of configuraties te reconstrueren wanneer dat nodig is.

5.2.1.4. Functionele eisen

De functionele eisen voor dit project zijn vrij simpel

- Een apart project dat je makkelijk kan refereren en makkelijk informatie kan ophalen
- De switch moet voor zo veel mogelijk met SNMP worden aangesproken
- De nodige attributen voor elke switchpoort zijn
 - Snelheid – de bandbreedte van de switchpoort in Mb/s
 - DuplexID – de duplexmodus van de switchpoort in een id van een enumeratie
 - Poe – de hoeveelheid wat er op dat moment op de switchpoort staat in WATT
- Een reset + een check status functie om een switchpoort te herstarten

5.2.1.5. Technische eisen

Aangezien dit project oorspronkelijk geen duidelijk afgebakende applicatielaag bevatte, heb ik de technische architectuur zelf opnieuw gestructureerd volgens een logisch 3-tier model. Hierbij werden de verschillende verantwoordelijkheden verdeeld over de data laag, business laag en presentatielaag, zodat het systeem beter onderhoudbaar en uitbreidbaar wordt.

5.2.1.5.1. Database

Voor dit project bestond de database reeds binnen de bestaande infrastructuur van het ziekenhuis. De basisstructuur was dus al aanwezig en werd in hoofdzaak hergebruikt. In het kader van deze opdracht zijn slechts beperkte aanpassingen uitgevoerd aan de datamodelstructuur.

Deze aanpassingen bestonden voornamelijk uit het toevoegen van extra velden om nieuwe functionaliteiten te ondersteunen, zoals bijkomende informatie rond devices en hun configuratie. Deze wijzigingen werden uitgevoerd door een medestagiair, in overleg met het bestaande datamodel en de noden van de applicatie. Hierdoor bleef de compatibiliteit met de bestaande systemen behouden, terwijl er toch voldoende flexibiliteit werd gecreëerd om de nieuwe functionaliteiten te ondersteunen.

5.2.1.5.2. Applicatie

Voor de applicatielaag werden een aantal duidelijke technische eisen vooropgesteld die noodzakelijk zijn om de functionaliteiten van het systeem correct te ondersteunen. De applicatie moest in staat zijn om netwerkgerelateerde informatie op te halen en beheertaken uit te voeren op een efficiënte en betrouwbare manier.

Een eerste belangrijke vereiste was de mogelijkheid om via SNMP (Simple Network Management Protocol) gegevens op te halen van netwerkapparatuur. Concreet gaat het hierbij om informatie zoals de snelheid van een switchpoort, de duplexmodus en de PoE-status (Power over Ethernet), indien deze beschikbaar is op het toestel. Deze gegevens zijn essentieel om een correct en actueel overzicht te krijgen van de status van de netwerkpoeën binnen het ziekenhuis.

Daarnaast moest de applicatie ook de mogelijkheid bieden om switchpoorten te resetten. Dit is een belangrijke beheertaak waarmee netwerkproblemen op afstand kunnen worden opgelost zonder fysieke interventie. Deze functionaliteit draagt bij aan een efficiënter beheer van de netwerkstructuur en vermindert de nood aan manuele tussenkomsten ter plaatse.

5.2.2. Realistatie

5.2.2.1. DataBase

	Column Name	Data Type	Allow Nulls
	ID	int	☒
	Naam	int	☒
	Opmerkingen	nvarchar(MAX)	☒
	SwitchID	int	☒
	VLAN	nvarchar(50)	☒
	FunctieID	int	☒
	Snelheid	decimal(18, 2)	☒
	DuplexID	int	☒
	Poe	decimal(18, 2)	☒
			☐

Figuur 20: Database tabel switchpoort

5.2.2.2. Uitdagingen

Net zoals bij de andere projecten heb ik ook binnen dit project verschillende uitdagingen ervaren. Door de technische aard en de focus op netwerkbeheer waren er enkele specifieke moeilijkheden die extra onderzoek en aanpassingen vereisten.

- **SNMP**
Binnen dit project heb ik gewerkt met SNMP (Simple Network Management Protocol), een technologie waarmee netwerkapparatuur kan worden uitgelezen. Dit was voor mij een volledig nieuw domein, waardoor ik in het begin veel tijd heb moeten besteden aan onderzoekwerk en testen om alles correct werkend te krijgen. Vooral het begrijpen van de structuur van SNMP-queries en de juiste OID's vergde de nodige tijd en experimentatie.
- **SSH**
Daarnaast heb ik ook SSH gebruikt om bepaalde acties uit te voeren op switches. Gelukkig had ik hier al beperkte ervaring mee, waardoor dit onderdeel vlotter verliep. Toch was er een belangrijke beperking: het bleek niet mogelijk om per individuele poort de PoE-status op te vragen via SSH, enkel per volledige switch. Om dit probleem op te lossen heb ik een helperfunctie ontwikkeld die deze informatie op een gestructureerde manier kon verwerken en koppelen aan de juiste poorten.
- **Rechten en toegangsbeheer**
Een bijkomende uitdaging waren de strikte toegangsrechten binnen de ziekenhuisomgeving. Het duurde enige tijd voordat ik de juiste accounts en permissies kreeg voor zowel SNMP- als SSH-toegang op elke switch. Dit was noodzakelijk om correcte testen uit te voeren en de functionaliteiten te kunnen ontwikkelen en valideren binnen een realistische omgeving.

5.2.2.3. Resultaat

Het resultaat van dit project is een aparte extensie op de bestaande Device Manager-applicatie. Deze uitbreiding maakt het mogelijk om, op basis van enkel een switchpoort-ID, automatisch alle relevante informatie op te halen en deze vervolgens, indien nodig, correct terug op te slaan in de database. Hierdoor wordt het beheer van switchpoorten sterk geautomatiseerd en vereenvoudigd.

Zelf ben ik tevreden met het eindresultaat van dit project, aangezien de applicatie op een gestructureerde en onderhoudbare manier werd opgebouwd. Er werd gewerkt met duidelijke scheiding tussen klassen voor SNMP- en SSH-functionaliteiten, waardoor de code overzichtelijk en herbruikbaar blijft. Daarnaast werd er bewust vermeden om hardcoded waarden te gebruiken, wat de flexibiliteit en veiligheid van het systeem verhoogt.

Ook werden gebruikersgegevens en wachtwoorden op een aparte en beveiligde locatie beheerd, wat bijdraagt aan een betere beveiliging van de applicatie. Verder is er aandacht besteed aan error handling, zodat fouten op een gecontroleerde manier worden opgevangen en verwerkt. Het geheel resulteert in een stabiele en goed werkende oplossing die aansluit bij de noden van de bestaande infrastructuur.

5.3. Chatbot

5.3.1. Analyse

5.3.1.1. Klant

Deze applicatie werd ontwikkeld als een proof of concept (POC) voor mijn stagementor, Stefan De Wilde. Het doel van deze opdracht was om te onderzoeken in welke mate het haalbaar is om intern een chatbot te ontwikkelen binnen de bestaande IT-omgeving van het ziekenhuis.

De stagementor wilde op basis van dit experiment evalueren of het realistisch is om een eigen chatbotoplossing uit te bouwen met interne middelen, of dat het in de praktijk beter is om hiervoor samen te werken met een externe partner. De applicatie dient dus voornamelijk als testcase om de technische mogelijkheden, beperkingen en onderhoudbaarheid van een interne chatbotoplossing te beoordelen.

5.3.1.2. Huidige situatie

Op dit moment is er binnen het ziekenhuis nog geen chatbot aanwezig. Wel bestaat er een centrale database met oplossingen voor veelvoorkomende IT-problemen. Deze database wordt via een bestaande applicatie geraadpleegd en bevat een verzameling van mogelijke incidenten en bijhorende oplossingen, zodat de helpdesk snel kan reageren op vragen en storingen.

Momenteel wordt deze kennisdatabank voornamelijk gebruikt door één persoon binnen de helpdesk. Hierdoor blijft de toegang tot de informatie beperkt en afhankelijk van deze gebruiker, wat de efficiëntie in sommige situaties kan verminderen. Wanneer deze persoon niet beschikbaar is, kan het langer duren voordat een oplossing gevonden wordt.

Het idee bestaat er daarom in om deze bestaande database te combineren met een chatbot. Op die manier zouden ook andere medewerkers binnen het ziekenhuis rechtstreeks vragen kunnen stellen aan het systeem en sneller antwoorden of mogelijke oplossingen kunnen terugvinden. Dit zou de toegankelijkheid van de informatie verhogen en de werkdruk op de helpdesk verlagen.

5.3.1.3. Probleem

Voor dit project is er op dit moment geen duidelijk of acuut probleem aanwezig, aangezien het voornamelijk dient als een proof of concept binnen het kader van mijn stage. Het project is eerder exploratief van aard en richt zich op het onderzoeken van de mogelijkheden rond de implementatie van een chatbot binnen de bestaande IT-omgeving van het ziekenhuis.

Het doel is niet om een bestaande fout of tekortkoming op te lossen, maar om na te gaan hoe een chatbot kan worden geïntegreerd met de reeds aanwezige kennisdatabank en welke meerwaarde dit kan bieden voor de gebruikers. De resultaten van dit project kunnen vervolgens dienen als basis voor een eventueel vervolgproject waarin de chatbot verder wordt uitgewerkt en effectief in productie kan worden genomen.

5.3.1.4. Functionele eisen

De functionele eisen voor dit project beschrijven welke functionaliteiten de chatbot moet aanbieden vanuit het perspectief van de eindgebruiker. Ze definiëren dus wat het systeem moet kunnen doen, los van de technische implementatie of de onderliggende technologieën.

Voor dit project bestaan de functionele eisen uit één kernfunctionaliteit. De chatbot moet in staat zijn om, op basis van een vraag van de gebruiker, relevante items uit de bestaande database op te zoeken en terug te geven. Hierbij wordt gebruikgemaakt van OpenAI om de vraag van de gebruiker te interpreteren en de juiste context of intentie te bepalen. Vervolgens worden de meest relevante resultaten uit de databank opgehaald en weergegeven aan de gebruiker, inclusief de bijhorende ID's.

Op die manier kunnen medewerkers snel en efficiënt de juiste informatie terugvinden zonder manueel door de volledige database te moeten zoeken.

5.3.1.5. Technische eisen

5.3.1.5.1. Chatbot

Voor de technische uitwerking van dit project werden enkele concrete eisen vooropgesteld met betrekking tot de werking van de chatbot. Een belangrijke vereiste was dat de chatbot geïntegreerd moest worden binnen een .NET Framework 6.2 Windows Forms applicatie. Binnen deze omgeving moest de chatbot in staat zijn om meerdere relevante items uit de database terug te geven op basis van een vraag van de gebruiker.

Aangezien mijn stagementor nog geen ervaring had met het gebruik van OpenAI voor het ontwikkelen van een chatbot, heb ik zelf het grootste deel van het opzoekwerk en de technische verkenning uitgevoerd. Dit omvatte onder andere het onderzoeken van de mogelijkheden van de OpenAI API en de manier waarop deze het best kon worden geïntegreerd in een bestaande .NET-omgeving. Dankzij eerdere ervaring die ik hierover had opgedaan tijdens mijn studies, kon ik dit proces efficiënter aanpakken en sneller tot een werkende oplossing komen.

5.3.2. Realisatie

5.3.2.1. Uitdagingen

Aangezien dit project beperkter was in omvang in vergelijking met de andere projecten, heb ik relatief weinig grote technische uitdagingen ervaren. Toch waren er enkele belangrijke aandachtspunten en leermomenten tijdens de ontwikkeling.

- **OpenAI-account en API-toegang**
Om gebruik te kunnen maken van de OpenAI API is een geldig account vereist, waarbij ook een betalingsmethode gekoppeld moet worden om de API te kunnen gebruiken. Gelukkig beschikte ik reeds over een bestaand account uit semester 1, waardoor ik dit onderdeel niet meer moest opzetten en direct kon starten met de implementatie.
- **System prompt**
Een tweede uitdaging was het correct opstellen van de system prompt. De OpenAI API maakt gebruik van een system prompt om het gedrag en de context van de chatbot te bepalen. Dit is in feite een set instructies die bepaalt hoe de AI moet reageren. Een eenvoudige wijziging, zoals “je bent een piraat”, verandert bijvoorbeeld volledig de stijl van de antwoorden. Het opstellen van een goede system prompt bleek echter minder eenvoudig dan verwacht, omdat deze zeer specifiek en duidelijk geformuleerd moet zijn om consistente en bruikbare resultaten te bekomen. Kleine aanpassingen in de formulering hadden vaak een grote impact op de kwaliteit en relevantie van de antwoorden, waardoor dit onderdeel een belangrijk iteratieproces vereiste.

5.3.2.2. Besluit

Dit project vond ik persoonlijk minder uitdagend en minder interessant in vergelijking met de andere projecten binnen mijn stage. Dit komt voornamelijk doordat ik gelijkaardige concepten al eerder heb behandeld tijdens mijn opleiding, waardoor de technische en conceptuele aspecten minder nieuw aanvoelden.

Daarnaast is de omvang van dit project relatief beperkt, waardoor er minder ruimte was voor diepgaande ontwikkeling of complexe architecturale keuzes. Het project was vooral gericht op het aantonen van de haalbaarheid van een chatbot binnen de bestaande omgeving, eerder dan op het uitwerken van een uitgebreid of volledig productiegericht systeem.

6. Besluit

Wanneer ik terugblik op mijn stage, kan ik stellen dat ik in zekere zin veel geluk heb gehad met de omgeving en de projecten die mij werden toevertrouwd. In plaats van één afgebakend stageproject, kreeg ik de kans om te ervaren hoe het is om te werken binnen een ziekenhuiscontext waar technologie voortdurend evolueert en waar steeds meer processen afhankelijk worden van digitale oplossingen. Hierdoor kreeg ik een breder inzicht in de realiteit van softwareontwikkeling binnen een kritische en professionele omgeving.

Tijdens mijn stage heb ik dan ook veel bijgeleerd, zowel op technisch als op professioneel vlak. Eén van de belangrijkste inzichten is het belang van code quality. Waar ik vroeger misschien meer gefocust was op het snel laten werken van een oplossing, heb ik geleerd dat onderhoudbaarheid, structuur en leesbaarheid minstens even belangrijk zijn, zeker in systemen die op lange termijn gebruikt en uitgebreid worden.

Daarnaast heb ik geleerd dat het essentieel is om kritisch te blijven nadenken over de oplossingen die je ontwikkelt. Het is belangrijk om niet zomaar aan te nemen dat de eerste aanpak de beste is, maar om voortdurend te evalueren of de gekozen oplossing wel degelijk de meest efficiënte en robuuste is. Deze kritische houding heeft mij geholpen om betere en meer doordachte software te schrijven.

LITERATUURLIJST

Gillis, A. S. (2024, October 22). *What is a 3-tier application architecture?* TechTarget.
<https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application>

Guru99. (2024, August 13). *N tier (multi-tier), 3-tier, 2-tier architecture with example.*
<https://www.guru99.com/n-tier-architecture-system-concepts-tips.html>

GeeksforGeeks. (2025, October 3). *Simple Network Management Protocol (SNMP).*
<https://www.geeksforgeeks.org/computer-networks/simple-network-management-protocol-snmp/>

Cisco. (n.d.). *Understanding Simple Network Management Protocol (SNMP).* Cisco Learning Network.
<https://learningnetwork.cisco.com/s/article/Understanding-Simple-Network-Management-Protocol--SNMP>

GeeksforGeeks. (2025, July 23). *SSH command in Linux with examples.*
<https://www.geeksforgeeks.org/linux-unix/ssh-command-in-linux-with-examples/>

6.1.1. Gebruik van artificiële intelligentie

Voor het opstellen van dit realisatiedocument heb ik gebruikgemaakt van artificiële intelligentie als ondersteuning bij het herschrijven en verbeteren van mijn eigen teksten. De AI werd hierbij uitsluitend ingezet als hulpmiddel om zinsbouw, grammatica en academische formulering te optimaliseren, zodat de inhoud duidelijker en professioneler geformuleerd werd. De oorspronkelijke inhoud en ideeën zijn steeds door mezelf aangeleverd en blijven volledig mijn eigen werk. De AI heeft dus niet bijgedragen aan het genereren van nieuwe inhoud, maar enkel aan het herstructureren en verfijnen van bestaande teksten om deze op een hoger academisch niveau te brengen.